

从 Agent 技术演进到 OpenCode 工程实践

面向软件开发人员的 Coding Agent 落地指南

过去：聊天机器人

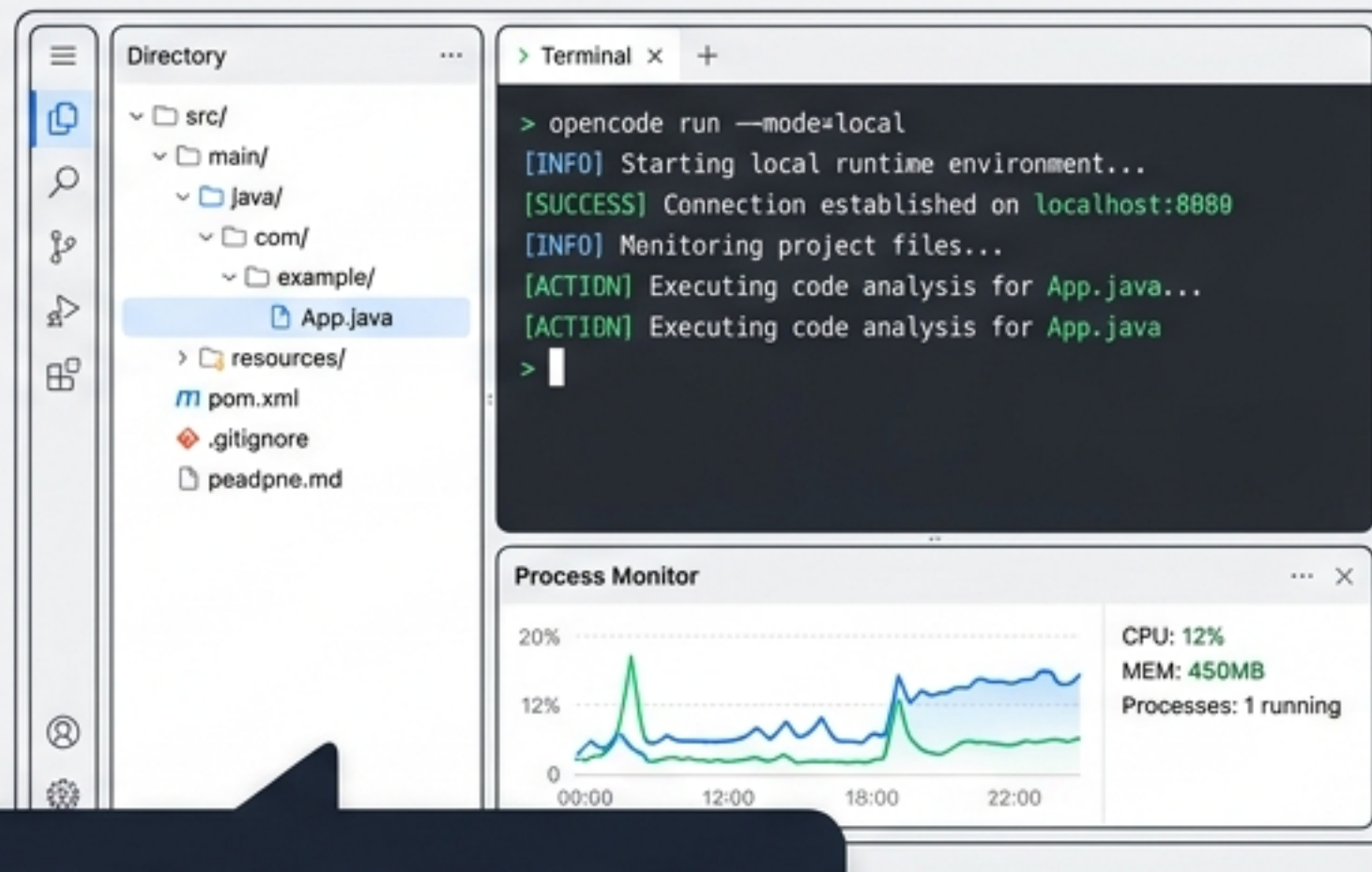
用户：如何写一个快速排序？

机器人：这里是一个快速排序的例子...

用户：如何写写写一个快速排序？

机器人：这里是一个快速排序的例子，可以是笨拙的快速排序。

现在：本地协作运行时



OpenCode 不是一个“会写代码的聊天机器人”，而是一个可观测、可配置、可治理的本地研发协作者运行时（Local Runtime）。

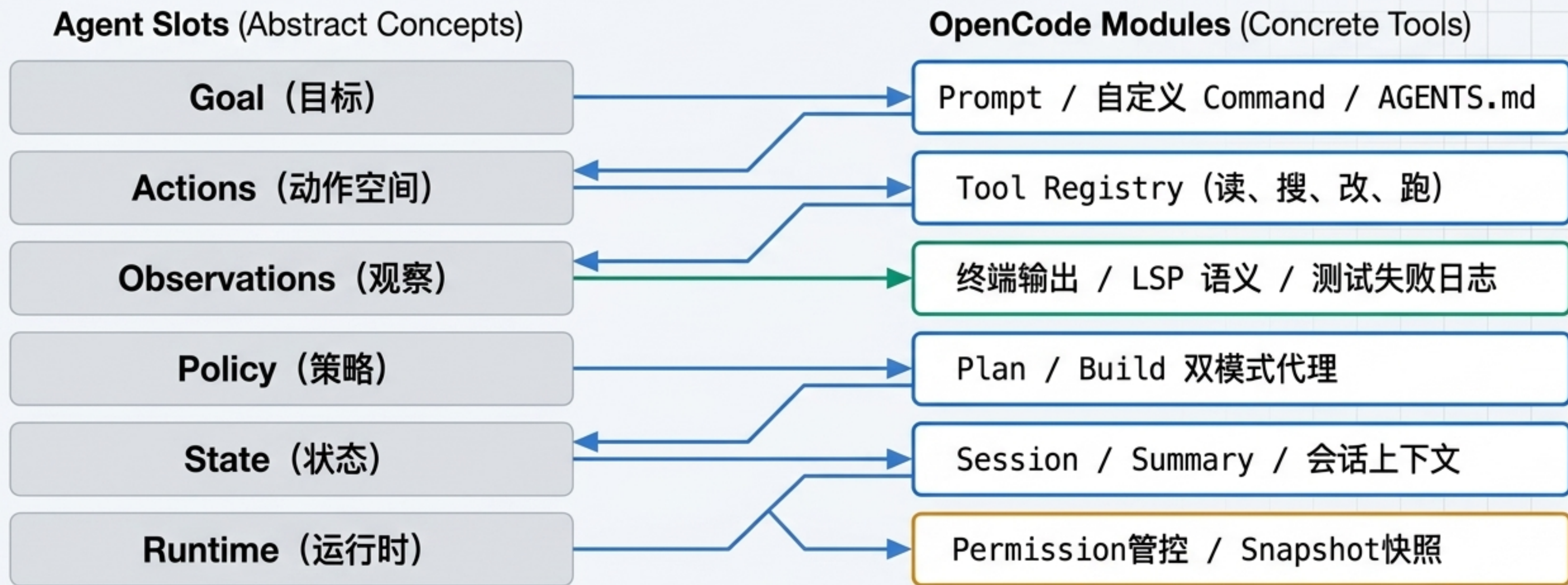
从“会想”到“能安全地改代码”



一个成熟的 **Coding Agent** = 模型 + 代码仓库上下文 + 工具动作空间 + 权限系统 + 会话状态 + 快照/回滚 + 团队规则

一个成熟的 **Coding Agent** = 模型 + 代码仓库上下文 + 工具动作空间 + 权限系统 + 会话状态 + 快照/回滚 + 团队规则

Coding Agent 的核心架构映射



ReAct 像“会自己排障的实习生”；OpenCode 像“带 IDE、终端、工单和权限审批的本地开发工位”。

双引擎驱动：先想清楚，再动手

Plan 模式

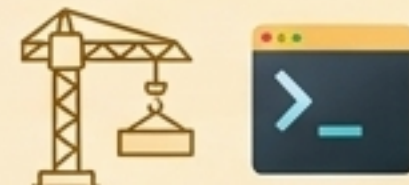


定位：受限代理，负责需求澄清、方案设计、风险分析。

约束：绝对不允许直接修改代码
(**edit = deny**)。

类比：架构设计阶段。

Build 模式



定位：默认开发代理，负责最小实现、补测试、跑验证。

约束：拥有完整工具访问能力，但受制于严格的权限拦截。

类比：编码与执行阶段。

SOP Rule

复杂任务永远遵循 **Plan** → 人类确认 → **Build** 的工作流。

像设计工程流一样设计 Prompt



帮我修一下登录报错的 bug。

诊断：太大、太模糊，一上来就让 Agent 自由发挥，容易引发代码破坏。



隔离执行

请只读探索，不要修改文件。
请分析 auth/ 目录下的报错原因。
输出相关文件、调用链和修复方案证据。

限制范围

强制收敛假设

核心原则：隔离实现与验证。不要让 Agent 一上来写代码，先让它收集证据、收敛假设。

AGENTS.md: 项目的标准作业指导书



这不是一段提示词，而是 OpenCode 理解项目的核心上下文。
每一次初始化 `/init` 后，第一步就是审查并完善这份工程治理文档。

权限治理：不要给 Agent 无限 Root 权限

● Allow (默认放行)

读操作可以宽泛

read grep glob

bash: git diff/status/log

● Ask (需人类授权)

写操作必须被监督

edit write apply_patch

● Deny (绝对拦截)

高危与破坏性操作拦截

rm -rf pkill

bash: git push/reset

只要这个命令在人类手上敲错都可能造成巨大损失，就绝不要让 Agent 自动执行。
规则后匹配优先，通配符 * 放前，精准限制放后。

状态管理与回滚协议

Git Checkpoint

绝对真实的结构化备份。这是主保险。

OpenCode /undo

局部的临时会话回滚。这是副保险。



操作规范

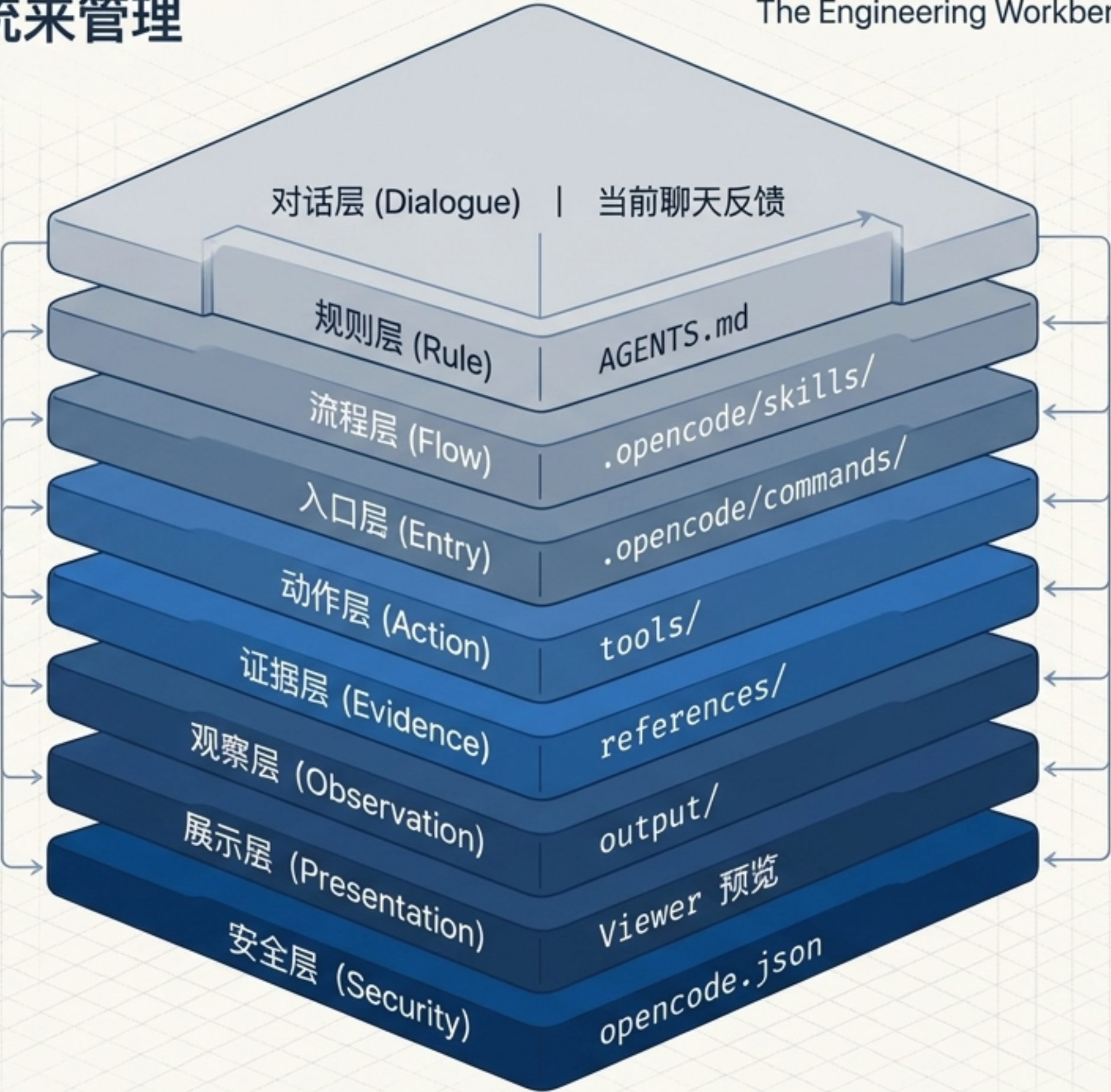
- ✓ Git 是主保险；/undo 仅为局部会话副保险。
- ✓ 每次大改前、切换子代理前、执行长命令前，必须手动通过 Git 做明确的 Checkpoint。

⚠ 风险警告

依赖缓存快照的 /undo 在多代理并行或跨文件系统操作时可能存在 stale lock 风险，切勿滥用。

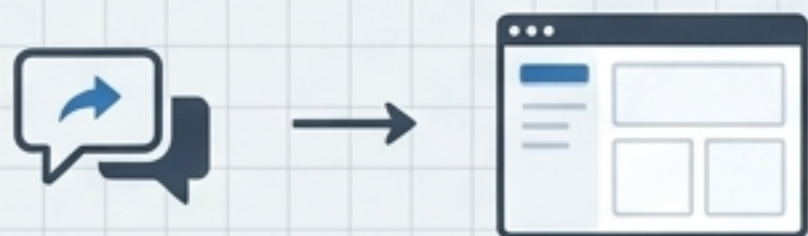
Context 分层架构：将上下文作为系统来管理

不要把上下文当成一个无限堆叠的 Prompt。要把它当当成一个可分层、可外部化、可交接的工程系统。



场景一：通过规则与规范约束 Agent

Demo 00: 规则驱动 Dashboard



动作:

一句话更新教学进展。

约束机制:

 固定模板

严禁模型自造 `HTML`。必须读取固定模板，通过内置脚本抽取出自参渲染。

结果:

每次生成的 UI 字段和样式绝对稳定，不漂移。

Demo 01: Web App 跨框架迁移



动作:

跨框架整体重构 (`Gradio` -> `Flask`)。

约束机制:

注入 功能 `Spec` + 前端风格 `Spec` + 历史参考代码。

结果:

不是简单的语法翻译，而是带有验收样例验证的工程级迁移。

Constraining the model yields predictable software engineering.

场景二：基于证据提取与受控沙箱的执行

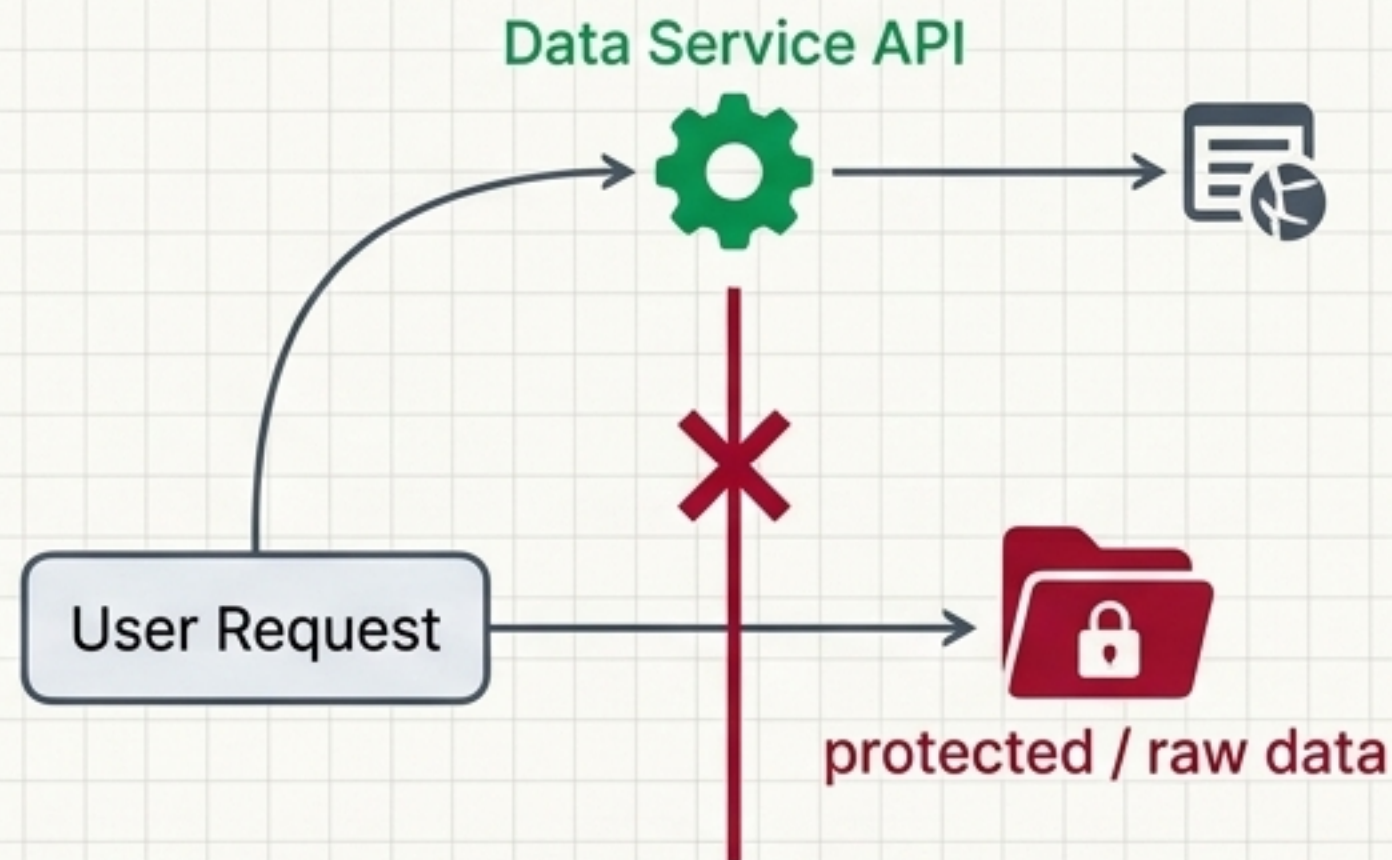
Demo 02: PDF 复现项目



核心认知:

必须先提取 Evidence, 让动作被证据约束, 而不是靠模型凭空“读完即写”。

Demo 03: 权限沙箱与数据服务



核心认知:

禁止直接访问敏感目录。生产中的 Agent 必须被封装在受控动作空间里, 仅通过安全封装的 API 获取聚合数据。

本地运行时的常见避坑与诊断

Alert Dashboard

Log 1: pkill 命令挂起 / 误杀进程

Root Cause: pkill -f 按命令行全匹配, 易杀掉 OpenCode 自身。



Fix: 在 `opencode.json` 中配置 `pkill \*` :
"deny"。



Log 2: Bash 工具因后台长任务无响应

Root Cause: Agent 运行常驻服务导致终端堵塞。



Fix: 强制 Agent 仅运行短命令 (lint, grep) , 长跑服务放回人类 `tmux` 管理。



Log 3: Context 极速膨胀

Root Cause: 开启过多非必要 MCP 工具或直接 write 覆盖大文件。



Fix: 默认关闭闲置 MCP, 优先使用 `Scout` 代理查阅文档, 修改优先使用 `apply\_patch` 或 `edit` 。



验证必须独立于实现：交接与审查循环



交接清单要求 (Handoff Summary Checklist)

- ☐ 定位到的根因是什么？
- ☐ 核心修改点在哪里？
- ☐ 补充了哪些测试用例？
- ☐ 本地验证命令的输出结果是什么？
- ☐ 存在哪些未覆盖的风险边界？

Golden Rule: 如果测试失败，Agent 必须先解释失败原因，绝对禁止在未明根因时扩大修改面。

驾驭 Coding Agent 的四个核心方法论

先设计动作空间，再谈自治

没有工具约束和权限边界的 Agent，只是在放大系统的不稳定性。

1

先取证验证纠偏，再自动完成

让动作被客观证据（Evidence）和严格的验收标准所强约束。

2

上下文是分层系统，不是巨型提示词

AGENTS.md、Skill、Tool 和 References 各司其职，外部化管理。

3

稳定性来自完整运行时，而非神奇 Prompt

生产级的可靠性来自于记忆、协议、工具、安全规则和工作流的精密配合。

4

研发团队起步的黄金路径

最终口诀：先规则，后任务；先计划，后修改；先小改，后测试；先 checkpoint，后放手。

低风险

Entry Point 1: 解释陌生模块

熟悉 `Explore` 模式，安全只读探索代码路径。

中风险

Entry Point 2: 补充单元测试

跑通 `Plan` 制定与测试验证的独立闭环。

高价值

Entry Point 3: 修复范围明确的小 Bug

跑通 `Build` 模式与 `Git` `Checkpoint` 安全流程。

掌握整个运行时的边界与协作节奏，再将核心代码库的钥匙交出。